DEVELOPMENT OF HINDI TO ISL TRANSLATION SYSTEM

Submitted in partial fulfilment of the Degree of

Bachelor of Technology

in

Computer Science Engineering / Computer Science-Artificial Intelligence

by

Avani Tank (2316363)
Riddhi Arya (2316076)
Riya Jolly (2316084)

Under the supervision of

(Dr. Nisheeth Joshi)
Professor
Department of Computer Science
Banasthali Vidyapith

DEPARTMENT OF COMPUTER SCIENCE

BANASTHALI VIDYAPITH, RAJASTHAN

2025-26

Certificate

This is to certify that the project work titled "**Hindi to ISL Translation System**" submitted by "**Avani Tank, Riddhi Arya, Riya Jolly**" in partial fulfillment for the award of degree of B. Tech in CSE/IT/CS-AI of Banasthali Vidyapith has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Dr. Nisheeth Joshi

Designation: Professor

Place: Department of Computer Science

Date: 04-04-2026

Abstract

Sign language is a way of communication using gestures and signs for the people belonging to the DHH (Deaf and Hard Of Hearing) community.The aim is to build a translation system that converts Hindi language to Indian Sign Language to enhance the communication capabilities of people with hearing and speaking disabilities. The existing systems primarily depend on sign to text conversion but the text to sign translation is limited. Although researchers have reached important milestones in developing automatic sign-language processing technologies, when it comes to ISL (Indian Sign Language), progress has been comparatively minimal.

The proposed system addresses this gap by providing a comprehensive and user-friendly solution capable of handling multimodal input. It integrates speech-to-text techniques to convert audio input into textual form, followed by NLP methods to analyze and preprocess the text. The processed input is then mapped to corresponding ISL gestures using a structured dataset. The output is presented in the form of animated visual signs, enabling users to easily understand and interpret the translated content.

Furthermore, the system is designed with scalability and adaptability in mind, allowing future enhancements such as real-time translation, improved contextual understanding, and integration with advanced machine learning models. It emphasizes accessibility, usability, and inclusivity, making it suitable for applications in education, healthcare, and public services.

By addressing the limitations of existing systems and focusing specifically on ISL with both text and audio inputs, this work contributes toward reducing communication barriers and promoting equal opportunities for the DHH community.

Acknowledgement

We take this opportunity to express our sincere gratitude to all those who have supported and guided us throughout the successful completion of our project titled *"Development of Hindi to ISL Translation System."* This work would not have been possible without the encouragement, guidance, and cooperation of several individuals and organizations.First and foremost, we would like to express our deepest gratitude to our esteemed project guide, **Dr. Nisheeth Joshi**, for his invaluable guidance, constant encouragement, and insightful suggestions throughout the course of this project. His expertise, patience, and continuous support played a vital role in shaping our understanding and improving the quality of our work. His constructive feedback helped us overcome challenges and successfully complete this project.We are deeply thankful to the **Department of Computer Science, Banasthali Vidyapith**, for providing us with the necessary academic support, resources, and a conducive learning environment. The department has always encouraged innovation and practical learning, which motivated us to explore new technologies and ideas. We also extend our gratitude to all faculty members for their indirect support and encouragement during our academic journey.We would also like to acknowledge **Banasthali Vidyapith** for offering us a platform that promotes academic excellence, discipline, and holistic development. The values and opportunities provided by the institution have played an important role in shaping our skills and approach towards learning and problem-solving.A special note of appreciation goes to the members of the deaf and hearing-impaired community who inspired the core idea behind this project. Understanding their real-world challenges helped us recognize the communication gap between Hindi speakers and ISL users. Their experiences guided us in developing a system that aims to promote inclusivity and accessibility. We hope that our work contributes, even in a small way, towards bridging this gap.We also extend our gratitude to the open-source community, researchers, and developers whose tools and technologies in speech recognition, natural language processing, and sign language animation greatly supported the development of this system. Their contributions made it possible for us to implement advanced concepts effectively.We would like to thank our peers and classmates for their encouragement, valuable suggestions, and constructive feedback, which helped us refine our ideas and improve the overall quality of our project.In conclusion, we are thankful to everyone who contributed to the successful completion of this project. This experience has been a valuable learning journey, enhancing both our technical knowledge and our understanding of the importance of inclusive and accessible technology.

Table of Contents

1. Introduction 6

2. Requirement Analysis (SRS) 7

2.1 Requirement Specification 7

2.2 H/w and S/w Requirements 11

2.3 Feasibility Study 12

2.4 Product Functions 13

2.5 Use-case Diagrams 15

3. System Design (SDS) 20

3.1 Higher-level Design 20

3.2 Class Diagrams 23

3.3 Data Design 23

3.4 Activity Diagrams 31

3.5 Sequence Diagrams 37

4. Coding (Modules) 39

5. Testing (Test Cases) 43

6. User Interfaces 46

7. Appendices 54

8. References 55

1. Introduction

Sign language is a vital mode of communication for individuals belonging to the Deaf and Hard of Hearing (DHH) community. It uses a combination of hand gestures, facial expressions, and body movements to convey meaning effectively. Unlike spoken languages, sign languages are visual in nature and have their own grammar and structure. Around the world, different regions have developed their own sign languages, which play a crucial role in shaping the culture and identity of deaf communities. Different regions of the world have their own version of sign languages like the American Sign Language (ASL), British Sign Language (BSL) and the Indian Sign Language (ISL), all representing their cultural and social identities of their respective deaf and hard of hearing community.

According to global estimates, millions of people rely on sign language as their primary means of communication. Even in India, a significant number of people face hearing and speech impairments, making accessibility tools a necessity rather than something luxurious. Despite this, communication barriers still persist between the DHH community and the general population because of lack of awareness and limited knowledge of sign language among hearing individual.

In India, **Indian Sign Language (ISL)** is widely used by the deaf and hearing-impaired population as their primary means of communication. ISL is a structured and expressive language that enables individuals to convey their thoughts, emotions, and information through gestures and visual cues. However, a significant communication gap exists between ISL users and individuals who primarily communicate through spoken or written Hindi. This gap often leads to challenges in everyday interactions, particularly in areas such as education, healthcare, workplaces, and public services.

Bridging this communication gap is essential for promoting inclusivity and equal opportunities among individuals. This also highlights the need for an automated, scalable and intelligent systems that facilitate seamless communication between Hindi speakers and ISL users.

The proposed system, *Hindi to ISL Translation System*, aims to address this issue by providing an automated solution that translates Hindi speech and text into ISL gestures. The system leverages modern technologies such as speech recognition, natural language processing, and animation models to convert spoken or written Hindi into a visual sign-based representation. By doing so, it facilitates better understanding and interaction between Hindi speakers and the DHH community. The Software Requirements Specification (SRS) document outlines the functional and non-functional requirements of the system, along with its scope, objectives, and constraints. It serves as a comprehensive guide for developers, designers, testers, and stakeholders involved in the project. The document ensures that all

requirements are clearly defined and understood, thereby enabling the development of an efficient, reliable, and user-friendly system.

Furthermore, the system focuses not only on the translation accuracy, but also on preserving the grammatical structure and contextual meaning of ISL, which differs significantly from the regularly spoken languages. This ensures that the generated output is not merely a word to word translation, but also preserves the correct grammatical structure.

The primary objective of this system is to promote accessibility and inclusivity by reducing communication barriers and enabling equal participation for individuals with hearing and speech impairments. Additionally, the system is designed to be scalable and adaptable, allowing future enhancements such as real-time translation, improved contextual understanding, and integration with advanced artificial intelligence technologies.

In addition, the system is designed with scalability and adaptability in mind. Future enhancements may include multilingual support, improved contextual understanding using advanced AI models. Such advancements will further enhance the systems usability and improve its impact across various domains.

2.Requirement Analysis

2.1. Requirement Specification

This section describes the detailed requirements of the Hindi to ISL Translation System. Each of the requirements is externally perceivable by the user, operator or other systems. The system is designed to ensure accurate, efficient and user-friendly translations from Hindi text or speech to the Indian Sign Language.

I. <u>Functional Requirements</u>

Input Processing

- The system shall input text in Hindi from the user.
- The system shall accept speech input in Hindi from the user through a microphone.
- The system shall validate the input to be Hindi language.
- The system shall preprocess the input by performing Tokenization (breaking sentences into words).
- The system shall provide real time feedback to the during the speech input like the recording status.

Translation Processing

- The system shall process Hindi text and speech input and convert it into its ISL representation.
- The system shall convert speech input into text using a speech to text module before further processing.
- The system shall use a predefined ISL dictionary containing mappings of Hindi words or phrases to ISL signs.
- The system shall apply ISL grammar rules for accurate translations that may differ from Hindi grammar.
- The system shall perform word to sign mappings for the words recognized.
- The system shall not process the words not mentioned in the dictionary.
- The system shall display a fallback message if no words match.
- The system shall map the words with their ISL counterparts.
- The system shall support basic sentence restructuring as per the ISL syntax i.e the Subject, Object, Verb format.

Output Generation

- The system shall process and display the ISL translations in the 2D animated form.
- The system shall provide ISL gloss based output with animated signs.
- The system shall synchronize output animations to ensure smooth and understandable gestures.
- The system shall allow the users to play, pause, replay and download the animations shown.
- The system shall ensure that the animations provided are visually clear, understandable and easy to interpret.
- The system shall provide word-wise animation output.
- The system shall support scalability for future 3D animation integration.
- The system shall display the output in a user-friendly interface.

Error Handling

- The system shall notify the user in case of empty or invalid inputs.
- The system shall display an arrow message if the input cannot be translated or any other fault.
- The system shall provide appropriate feedback if the speech input is unclear or not recognized.
- The system shall allow the user to retry input in case an error occurs.
- The system shall ensure that system crashes or failures are minimized through proper exception handling.

User Interaction

- The system shall provide a simple and intuitive user interface.
- The system shall support both beginner and experienced users.
- The system shall ensure accessibility for users with disabilities.

II. Non functional Requirements

AVAILABILITY

- The system shall be available at all times, subject to internet availability and server uptime.
- The system shall ensure high availability with minimal downtime.
- The system shall be available fully to the registered users while partially available to the non-registered users.
- The system shall notify users in advance regarding scheduled maintenance or downtime.

SECURITY

- The system shall authenticate the users at the time of login with username and password.
- The system shall authorize the users based on their roles like admin, registered user or guests.
- The system shall block all unauthorized user attempts to access the system.

RELIABILITY

- The system shall ensure translation accuracy remains consistent across sessions.
- The system shall handle unexpected errors or failure gracefully without crashing or causing inconvenience.
- The system shall provide stable performance under normal loads or peak loads.
- The system shall maintain uptime consistency and minimize failure rates.

PERFORMANCE

- The system shall process and display translations within acceptable response time.
- The system shall support real-time or near real-time translations for speech input.

- The system shall handle multiple user requests simultaneously without significant performance degradation.
- The system shall optimize animation rendering speed for smooth playback.
- The system shall efficiently manage memory and CPU utilization.

USABILITY

- The system shall provide a simple, user-friendly interface.
- The system shall include clear instructions and guidance for first time users.
- The system shall ensure accessibility for users with disabilities like clear visual outputs, easy navigation etc.
- The system shall maintain consistent UI design across all pages.

PORTABILITY

- The system shall be platform independent accessible via browsers.
- The system shall have a responsive design i.e. adapt to different screen sizes.
- The system shall not require installation for basic usage. The user shall be able to use it via a browser.
- The system shall be deployable on different operating systems without modification.

MAINTAINABILITY

- The system shall have clear and well documented code for easy understanding and further updations.
- The system shall follow coding standards and best practices.
- The system shall allow updates to the ISL dictionary without affecting other modules.
- The system shall have separate front-end and back-end logic to simplify maintenance.
- The system shall allow easy addition of new features and updation of the existing ones without the need to rewrite the existing system.
- The system shall ensure easy debugging and troubleshooting.
- The system shall ensure version control.
- The system shall allow any addition of new features without affecting existing functionalities.
- The system shall provide testing support to ensure new updates do not break any of the existing functionalities.

SCALABILITY

- The system shall be scalable to handle any increase in user visiting the website or data.
- The system shall support future enhancements such as multilingual support, AI-based contextual understanding.

COMPATABILITY

- The system shall be compatible with different browsers and devices.
- The system shall integrate with external APIs such as speech recognition services.
- The system shall support standard data formats for input and output.

2.2. Hardware and Software Requirements

Hardware Requirements:

- A microphone device capable of capturing clear Hindi speech input for speech recognition.
- A display screen for rendering ISL gesture animations with adequate resolution to ensure sign clarity and facial expression visibility.
- Standard input/output devices (keyboard, mouse, touchscreen) to facilitate user interactions and feedback submissions.
- Server Infrastructure:

  CPU: Dual AMD Rome 7742, 128 cores total, 2.25 GHz

  RAM: 1 TB

  Storage: 15TB (4x 3.84TB) U.2 NVME drives

  GPU : 8x NVIDIA A100 Tensor Core GPUs

Software Requirements:

- Operating Systems: Windows, Linux
- Frontend Technologies: HTML, CSS3, python
- Backend Framework: Python

- AI/ML Tools: NLP models, M2M100(Many-to-Many multilingual translation model), SentencePiece Tokenizer
- Libraries: NumPy, Pandas
- Browser Support: Chrome, Firefox, Edge

2.3. Feasibility Study

The feasibility study evaluates the practicality and viability of the Hindi to ISL Translation System from multiple perspectives including technical, operational, economic and social aspects.

Technical Feasibility

- The system uses established technologies such as **speech recognition, NLP, and AI-based animation**, making it technically feasible.
- Availability of powerful hardware (GPUs) ensures real-time processing.
- Integration of existing models like M2M100 reduces development complexity.
- Existing libraries and APIs further simplify the system implementation.
- The system supports scalability and future enhancements.

Operational Feasibility

- The system is designed for ease of use with a simple and accessible interface, ensuring ease of use for both technical and non-technical users.
- It requires minimal technical knowledge, making it suitable for end users, NGOs and educational institutes.
- The system can be accessed via web browser eliminating the need for complex installations.
- It supports real-time interactions, enabling smooth communication between users.
- The system can effectively be used in real-world environments such as classrooms, government offices, hospitals and workplaces.
- Basic user support and guidance ensure smooth adoption and usability of the product.

Economic Feasibility

- Use of open-source tools and frameworks reduces development and licensing costs. Thus, making the system economically feasible.

- Cloud-based deployment allows scalability with manageable expenses.
- The use of minimal hardware requirements for end users reduces overall implementation cost.
- Maintenance costs are manageable due to modular system design.
- The system has potential for future monetization or funding support through government initiatives, social impact programs and institutional collaborations.

Social Feasibility

- The system promotes inclusivity by enabling communication for the DHH community and reducing gap between them and the Hindi speakers.
- It is beneficial in sectors like education, healthcare, workplaces, and public services.
- It empowers individuals of the DHH community to communicate more independently and actively participate in society.
- The system has significant impact in sectors such as education (inclusive classrooms), healthcare (doctor-patient communication), employment (workplace inclusion) and public services (government bodies).
- It contributes to digital empowerment and social equality.
- The system aligns with the initiative of promoting accessible technology and inclusive development.

2.4. Product Functions

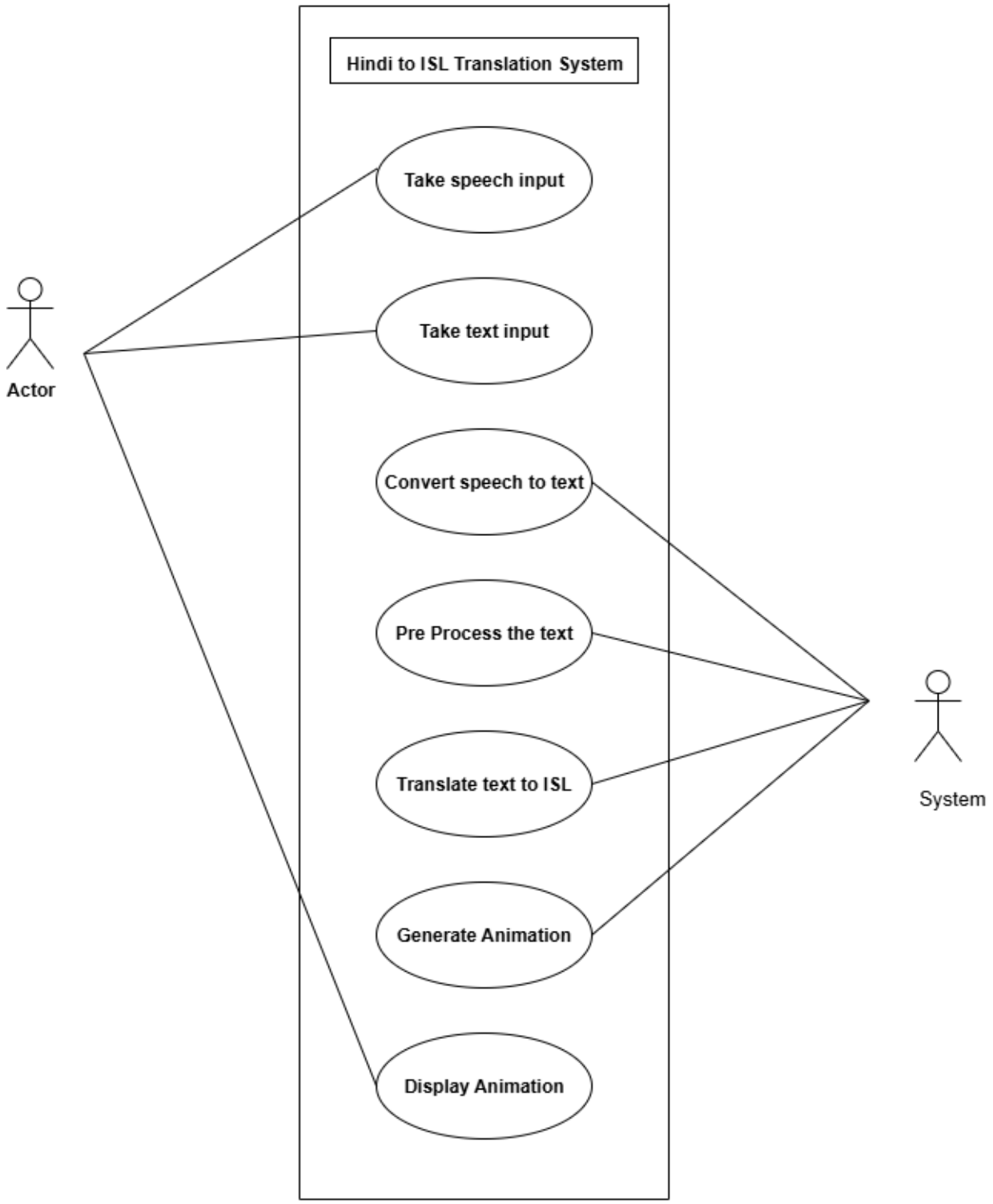
The major functions of the system are:

1. **Speech Recognition:**
    - Recognize Hindi speech and convert it into accurate written text.
    - The system shall capture Hindi speech input through a microphone.
    - It shall convert speech into accurate textual format using speech recognition techniques.
    - It shall support real time processing.
    - The system shall handle accent, pronunciation and speech speed.
2. **Text Processing:**
    - Handle grammar, context, and meaning of Hindi text to prepare it for ISL translation.
    - The system shall process Hindi text to extract meaningful linguistic information.
    - It shall perform tokenization and normalization.
    - The system shall analyze grammar, syntax and context of the input.

- The system shall restructure the input sentences according to the ISL grammar rules.

3. **Hindi-to-ISL Translation:**
    - Map Hindi words/phrases into ISL grammar and signs.
    - The system shall convert the input hindi sentence to english.
    - The system shall then map the English words and phrases to their corresponding ISL signs.
    - The system shall use a predefined ISL dictionary along with grammar rules.
    - The system shall handle unknown words and context-based translations.
    - It shall ensure that translations are meaningful and not merely just word-to-word conversions.
    - The system shall maintain consistency in translation output.
4. **Sign Representation:**
    - Display ISL signs through a 2D animated character.
    - The system shall display ISL output using a 2D animated character or avatar.
    - It shall generate clear and understandable gestures.
    - The system shall ensure smooth animation transitions and proper synchronization with the gloss outputs.
    - It shall show playback controls such as play, pause, replay and download.
    - The system shall support future upgrades to 3D animation or advanced visualization.
5. **User Interaction:**
    - Provide a simple, accessible interface for end users (deaf, hearing-impaired, NGOs, etc.) to interact with the system.
    - The system shall provide a simple and accessible user interface.
    - It shall allow users to enter text or speech input, view translated output and control animation playback.
    - The system shall support different user groups, including the DHH community, general users , NGOs and institutions.
    - The system shall ensure accessibility and ease of navigation.

2.5. Use-case Diagram



Description of Use Case Activities:

1. Take Speech Input

Take Speech Input	Details
Input	User speaks in Hindi (audio)
Processing	A microphone records and captures an audio signal
Output	Hindi audio data
Alternate(Error Handling)	If speech is not captured (noise, no mic) → system switches to text input

2. Take Text Input

Take Text Input	Details
Input	User types Hindi text
Processing	System captures and stores text
Output	Hindi text
Alternate (Error Handling)	If text invalid/unsupported → request re-entry or correction

3. Convert Speech to Text

Convert Speech to Text	Details
Input	Hindi audio data
Processing	Apply ASR model to transcribe spoken Hindi to text
Output	Transcribed Hindi text
Alternate(Error Handling)	If recognition accuracy is low → request confirmation or allow correction

4. Pre-process the Text

Pre-process the Text	Details
Input	Raw Hindi text
Processing	Clean text (remove noise), normalize words, tokenize
Output	Structured Hindi text
Alternate (Error Handling)	If ambiguity/error → prompt user for clarification

5. Translate Text to ISL

Translate Text to ISL	Details
Input	Pre-processed Hindi text
Processing	Maps Hindi grammar/words to ISL dictionary, adjust syntax, handle idioms
Output	ISL gloss/notation form
Alternate(Error Handling)	If word not available → fallback to fingerspelling/approximation; Show partial translation with highlights

6. Generate Animation

Generate Animation	Details
Input	ISL text/sign sequence
Processing	Animation module generates signing animated gestures
Output	ISL animated gestures
Alternate (Error Handling)	If sign animation missing → use static image/text description

7. Display Animation

Display Animation	Details
Input	Generated ISL animated
Processing	Render animation on screen in real-time
Output	ISL animation shown to user
Alternate(Error Handling)	If device cannot play animation → show frame-by-frame images or text

3. System Design(SDS)

3.1. Higher-level Design

The system is composed of several interconnected components that work together to translate Hindi into ISL animations. The interaction begins at the user-facing web client, flows through our backend server, leverages a powerful external AI service, and returns the visual translation to the user. The Hindi to ISL Translation follows a multi-layered architecture which is designed to ensure scalability, modularity and efficient processing of complex tasks such as speech recognition, natural language processing and animation rendering.

1. **Presentation Tier (Frontend)**

- Developed using **Streamlit.**
- Provides a user-friendly interface for input and output
- Allows users to:
    - Enter Hindi text
    - Provide speech input via microphone
    - View ISL animations
    - Login
    - Register

These components interact seamlessly to convert Hindi speech and text into meaningful ISL 2D animations.

Key Functionalities:

- Accept Hindi input through a text area.
- Capture speech input using a microphone.
- Displays recognized speech text, ISL gloss output and the animated video output.
- Provides real-time feedback such as processing indicators, error messages and success notifications.
- Allows user to control functionalities such as re-entering input, triggering translation and viewing results interactively.

Design Considerations:

- Clear visual hierarchy for better readability.
- Accessibility for DHH users through visual emphasis.

2. **Application Tier (Backend)**

- Developed using **Python**
- Acts as the central processing unit of the system
- It handles all logical operators and coordinates communication between frontend and AI-ML services.
- Responsibilities include:
    - Handling API requests
    - Processing input data
    - Performing text preprocessing and tokenization
    - Communicating with AI/ML services
    - Handling user requests from the frontend.
    - Managing input processing of text and speech.
    - Performing sentence segmentation and tokenization .
    - Executing translation pipeline from Hindi to English and then English to ISL Gloss.
    - Managing ISL Dictionary or the lexical-based mapping.
    - Managing error handling and response generation.

Key Modules:

- Speech Processing Module: Converts audio input into text.
- Text Processing Module: Cleans and prepares input.
- Translation Module: Converts the entered Hindi text to English and then to the corresponding ISL gloss.
- Video Processing Module: Merges sign videos into a final animation.

Design Consideration:

- Modular design ensures easy maintenance and scalability.
- Independent modules allow future upgrades without affecting the entire system.

3. **Service Tier (AI/ML Layer)**

- This layer is responsible for integrating machine learning and NLP models to perform intelligent translation.
- It also acts as the brain of the system and is responsible for understanding and transforming language.

Technologies used:

- Uses **M2M100 model** for multilingual translation.
- NLP Models for linguistic processing
- **ISL lexicon and grammar rules for structured sign generation.**
- **The dataset contains pre-defined animated videos for sign representation.**

Responsibilities:

- Responsible for:
    - Mapping Hindi text to ISL gestures.
    - Generating 2D animated sign language output.
    - Converting Hindi text into correct English representation.

4. **<u>Working Flow</u>:**

User Input → Frontend → Backend Processing → Speech Recognition → Translation → ISL to Gloss Conversion → Video or Lexicon mapping → Animation Merging → Display

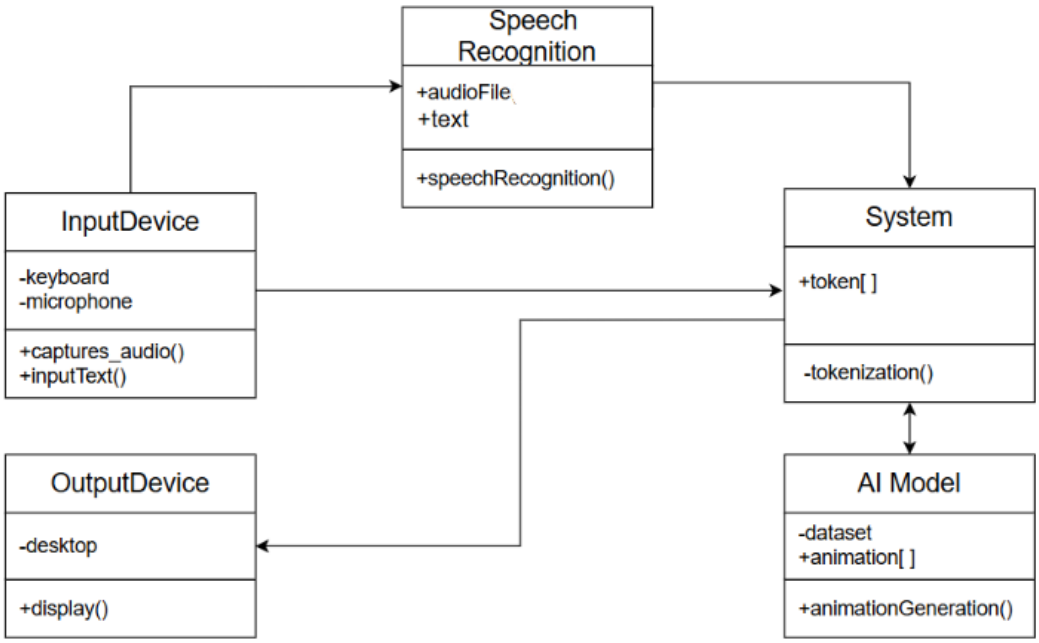
This layered design ensures efficient handling of complex operations and improves system performance.

5. **System Characteristics**

- Modularity: Each component operates independently.
- Scalability: Supports addition of new features and models.
- Flexibility: Can integrate new datasets and AI models.
- Efficiency: Optimized pipeline for faster response time.
- Maintainability: Clear separation of concerns.

6. Advantage
    - Reduces system complexity through layered architecture.
    - Improves performance and scalability.
    - Enables parallel development of modules.
    - Facilitate easy debugging and testing.
    - Supports future expansion.

3.2. Class Diagrams:



Detailed Class Explanations

Microphone

- **The microphone is** responsible for capturing spoken hindi input from the user. It aslo acts as a primary interface for voice recognition.
- Role: Captures spoken Hindi input.
- Methods:
    - captures_audio(): Records audio input for further processing.
- Interaction: Feeds raw audio data to the SpeechRecognition class for transcription.
- Works closely with the frontend to trigger recording events.
- This ensures real-time voice input collection which is crucial for enabling accessibility for users who prefer speaking over typing.

Keyboard

- It handles manual input of Hindi text entered by the user.
- Role: Allows manual input of Hindi text.
- Methods:
    - input_text(): Accepts user-entered Hindi text.
- Interaction: Sends textual data for tokenization.
- It interacts directly with the frontend UI.
- This provides an alternative input method, ensuring flexibility for users who prefer typing over speaking.

SpeechRecognition

- It basically converts spoken Hindi audio to textual format using speech recognition techniques.
- Role: Converts spoken Hindi audio into text format.
- Attributes/Methods:
    - speechFile / text: Interfaces for file handling.
    - SpeechRecognition(): Method performing the recognition/transcription.
- Interaction: Receives audio from Microphone, outputs recognized Hindi text to System.
- It acts as a bridge between audio input and textual processing.
- It plays a key role in enabling speech-based interaction and improving accessibility.

System

- The system class acts as the central controller and processing unit.
- It manages processing, tokenization and overall workflow.
- Role: Central processing unit for text handling and linguistic tokenization.
- Attributes/Methods:
    - token[]: Stores processed tokens/words.
    - tokenization(): Performs tokenization of input Hindi text.
- Interaction: Receives text input from SpeechRecognition or Keyboard, passes tokenized output to the AI Ml.
- It prepares data for translation.
- It ensures that the input is clean, structured and ready for further NLP operations.

AI Model Mapping

- Role: Core module for generating ISL animations from tokenized Hindi text.
- Attributes/Methods:
    - dataset: Contains mappings or models for Hindi-to-ISL translation.
- animation[]: Stores generated ISL animation frames or data.
    - animationGeneration(): Converts tokens into corresponding ISL signs/animations.
- Interaction: Receives tokens from System, outputs generated animation data to Desktop.
- This class performs the intelligent transformation of language, combining NLP and rule-based processing.

Desktop

- Role: Final output interface, presenting ISL animation to the user.
- Methods:
    - display(): Shows the generated ISL animation.
- Interaction: Receives ISL animation data from AI Model.

Process Flow Summary

- The user provides Hindi input via Microphone (speech) or Keyboard (text).
- Speech input is converted to text by SpeechRecognition, both sources route data to the System.
- The system tokenizes the Hindi text for efficient mapping.
- Tokenized data is passed to the AI Model, which generates corresponding ISL animations.
- Animations are sent to the Desktop for user display.

3.3. Data Design

Data structures

The system does not incorporate a relational database; rather, it utilizes structured temporary and internal datasets. The iSign dataset serves as the core data source, providing sampled pairs for Hindi phrases/sentences and corresponding ISL gesture information.

The system uses the pre-defined dataset. And lexicons as its core data source, which are loaded into memory during runtime for efficient access.

Internal Data Structures:

- Token List: Stores processed tokens extracted from input Hindi sentences for mapping.
- Mapping Table: In-memory dictionary or array linking Hindi words/phrases to ISL signs or glosses.
- Animation Frame Buffer: Sequence or queue storing ISL animation data for rendering.
- Temporary Input Buffer: Holds interim user input—speech or text—during preprocessing.
- Example:
    - Input: "मैं स्कूल जा रहा हूँ"
    - Token List: [मैं, स्कूल, जा, रहा, हूँ]

Temporary Data Structures:

- Preprocessing Buffer: Contains cleaned and normalized text/speech prior to ISL mapping.
- Error Flag/Status Object: Captures fault conditions (invalid input, mapping errors) in workflow.

Each of these structures exists transiently for the duration of processing a single user request and is destroyed after the transaction. No persistent storage or external database layer is present.

Files (Interim Use):

- Speech/Audio Cache: Temporary audio files generated when users record speech input, deleted post-processing.
- Animation Render Objects: Generated 2D animation files/images, stored briefly for display before being removed.

Database Description:

The Hindi to ISL translation system uses SQLite, which is a serverless and self contained relational DataBase engine. The DataBase file is named "form.db"

Database Schema:

Column Name	Data Type	Description
email	TEXT	Primary Key. Unique identifier for each user
password	TEXT	(SHA-256) Hashed password string
name	TEXT	Full name
gender	TEXT	Male/ Female/ Prefer not to say
dob	TEXT	Date of Birth
phone	TEXT	10-digit contact number
language	TEXT	User's preferred spoken language (Hindi/ English)
notif	TEXT	Communication preference (Email/ SMS/ None)

isl_speaker	**TEXT**	**for ISL knowledge (Yes / No)**
isl_exp	**INTEGER**	**Years of experience in Indian Sign Language**
isl_level	**TEXT**	**Proficiency level (Beginner to Fluent)**
bio	**TEXT**	**Optional description**

Data Schema Table

The users table is designed to store both authentication data that includes Full name, email id, password(hashed) along with user-specific preferences for the Indian Sign Language (ISL) translator.

Note; Primary Key: The email field serves as the Primary Key, that ensures no two accounts can be created or generated using the same email address.

2. Security Implementations

- **Hashing of Password:** the form does not store text password, it uses SHA-256, that is a well known and popular hashing algorithm used for encryption and also used in BlockChain.
  SHA 256 converts the password into a 64 character Hexadecimal string.
  It ensures the safety of pass-word even if the database is accessed.

- **Thread Safety:** The connection established between Database and webpage uses check_same_thread=False, which is crucial for Streamlit because Streamlit web framework handles multiple user sessions on different threads at the same time.

Workflow of User Authentication (LOGIN & REGISTRATION):

Registration: for registration process, the system validates the primary key and new not null features, Including- Full name, Gender , DOB, Email id (valid format), phone number (must be 10 digit), the password , that must be verified but typing it twice and then it is processed using SHA 256 hashing algo before writing it to Database.
We have also included the feature of terms and conditions to make it appear ready to launch. Other attributes include- experience in ISL, optional bio and preferred language .

Login: During login, the entered password is hashed and compared against the stored hash in the users table. If the hashes from the database match and the email exists in users table, a session is created using Streamlit's **session state**.
An optionality of logging out is also provided to the user to further join from another account or continue without an account.

Database Initialization Algorithm:

Step 1:Start the application.

Step 2:Identify the directory where the program file is located.

Step 3:Create the path for the database file (form.db).

Step 4:Establish a connection to the SQLite database.

Step 5:Create a cursor to execute SQL commands.

Step 6:Check if the users table exists.

Step 7:If the table does not exist, create it with all required fields (email, password, name, etc.).

Step 8:Save (commit) the changes to the database.

Session Management:

Step 1:When login is successful:

- Store user information in session state.

Step 2: While session is active:

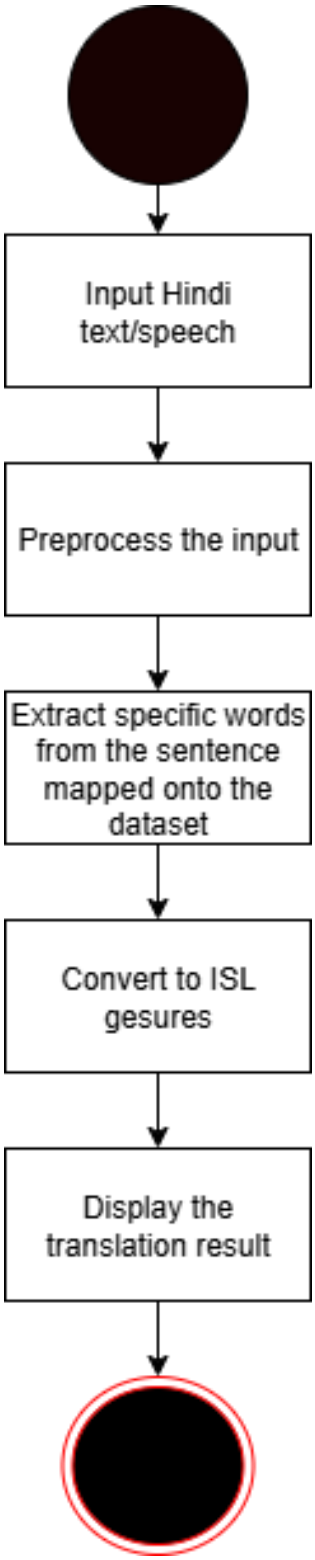
- User remains logged in

Step 3: When user clicks logout:

- Clear session data.
- Return to login page.

3.4. Activity Diagram

Overall Flow:



Description of the mentioned components in brief:

Start Translation:

- This is the entry point where the user starts the translation system.
- It is triggered when the user clicks on the "Translate" button or begins inputting as speech into the system.
- It ensures that the system is ready to capture and process the input.

Input Hindi text/speech:

- The system shall accept Hindi input from the user
- Input is accepted in 2 forms:
    1) Text Mode: User types a sentence in Hindi.
    2) Speech Mode: User speech is captured via a microphone.
- The system stores the input temporarily for processing.

Processing the input:

- The system cleans and pre-processes the input for translation.
- Steps include:
    1) Removing punctuation, special characters etc.
    2) Handling different forms of the same word.
- The steps ensure that the input is in standard format suitable for analysis.

Extract specific words from the sentence mapped onto the dataset:

- The system breaks the input sentence into single words known as tokenization.
- Each word is matched with the entries in the ISL dictionary.
- If a word is not found, the system:
    1) Uses stemming to find base forms.
    2) Uses synonyms or approximate matches.
- The system produces a sequence of valid ISL animations as the output.

Convert to ISL gestures:

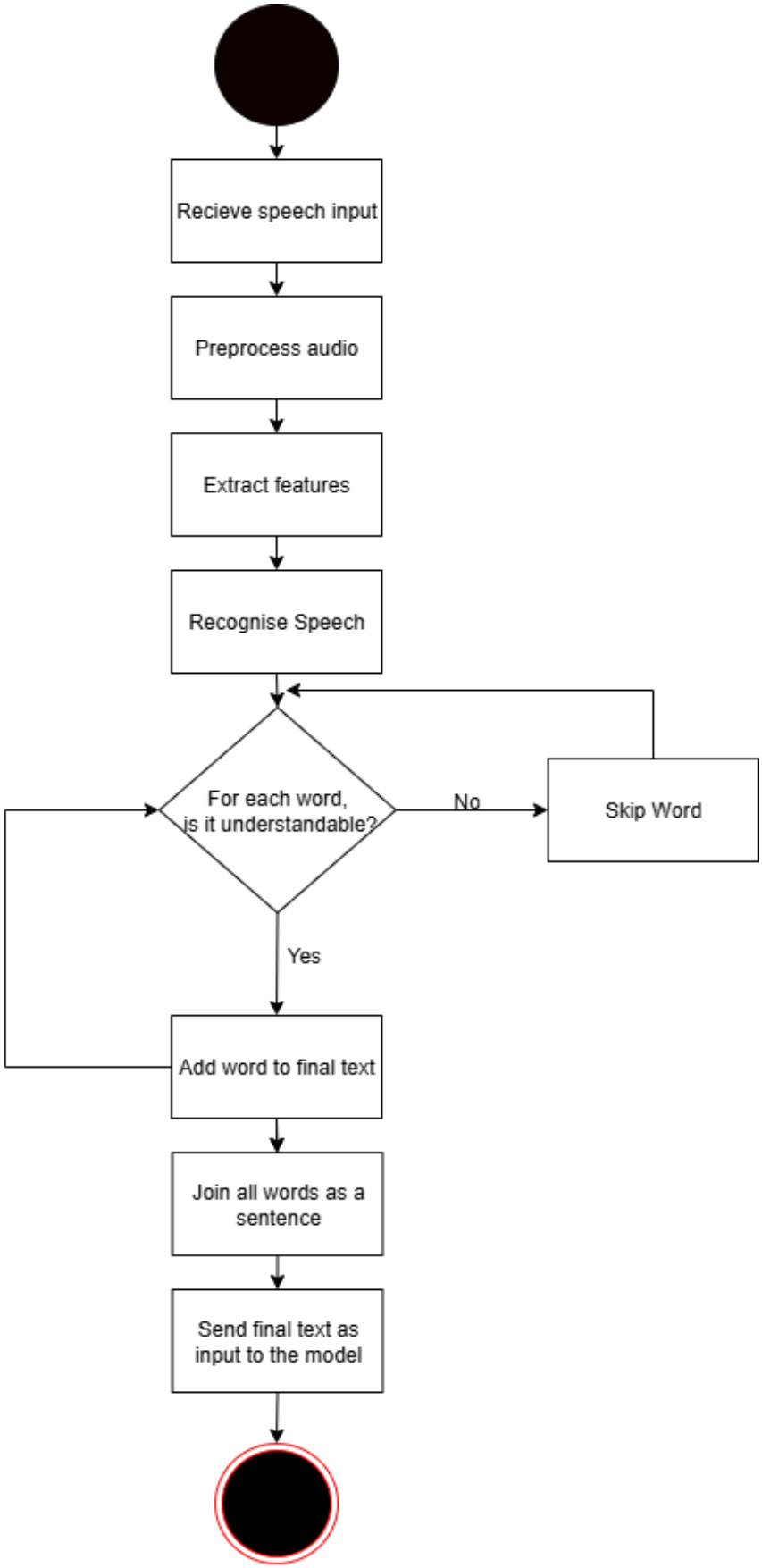
- The mapped words are now converted to their corresponding ISL gestures.
- A call is sent to Ham2Pose.
- It then returns the mapped 2D animation.
- This step is the actual translation from the textual data into visual sign language.

Display the Translation result:

- The final returned ISL gesture output is displayed to the user.

- This completes the translation and provides visual feedback to the user.

ASR to Text:



Description of the mentioned components in brief:

Start:

- This is the entry point where the user inputs the speech.
- It is triggered when the user clicks on the mic button as it begins inputting into the system.

Receive speech input:

- Captures raw data for processing.
- The user inputs speech or provides an audio file.

Preprocess audio:

- Improves audio quality to make speech clearer.
- Tasks may include:
    1) Noise Reduction
    2) Volume Normalization
    3) Removing Silence or Irrelevant Parts

Extract Features:

- It makes audio ready for recognition.

Recognise Speech:

- Uses Automatic Speech Recognition Model (ASR) to convert the extracted features into words.
- Outputs relevant words.

For each word, Is it understandable?:

- It checks if the word is clearly recognized or not.
- Decision Point:
    1) Yes → Word is understandable.
    2) No → Word is skipped

Skip Word:

- If a word is unclear or not recognized, it is ignored.
- Prevents errors.

Add word to final text:

- Adds correctly recognized words into the final text list.

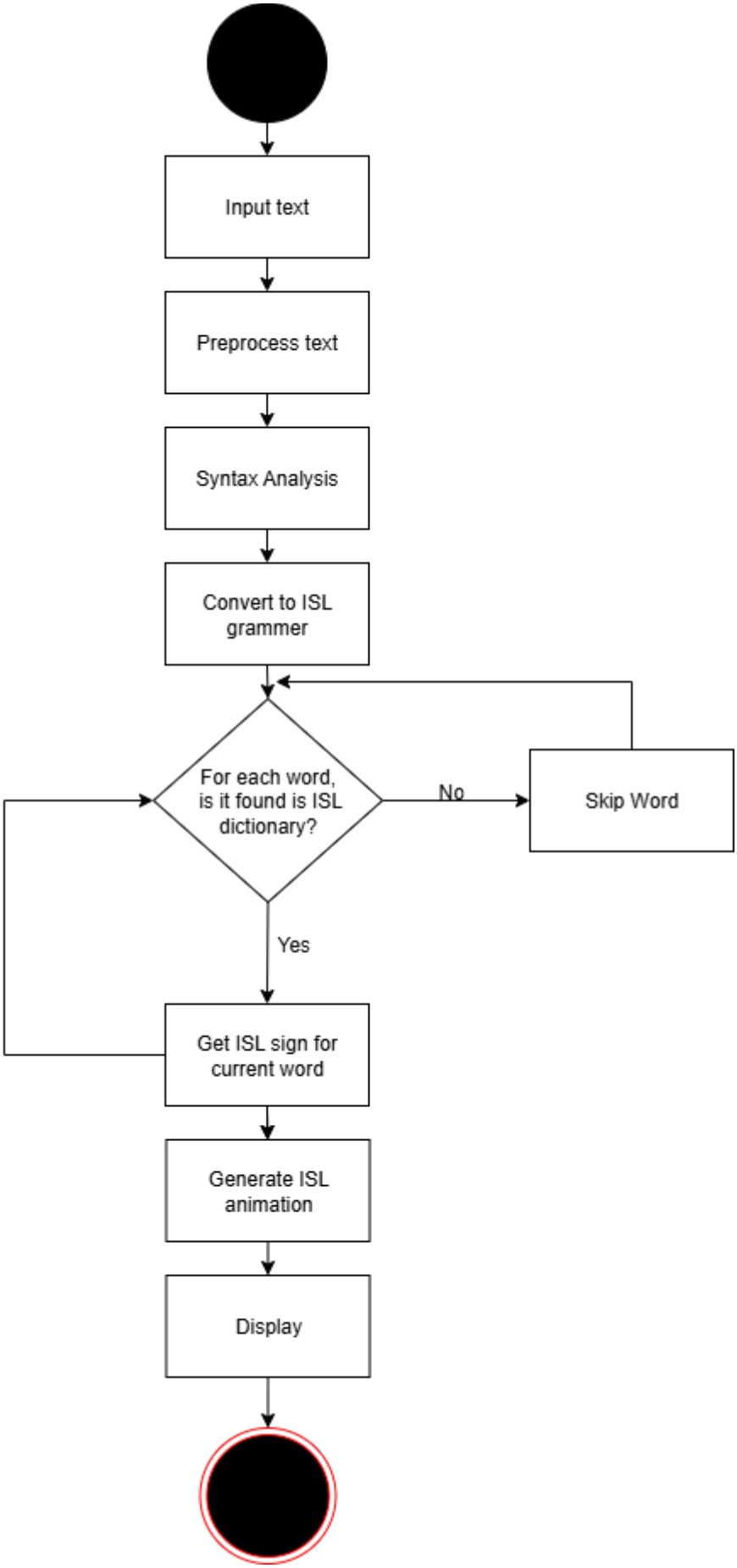
Join all words as a sentence:

- Join all accepted words to make a complete sentence.
- Skips any part not relevant.

Send Final Text as Input to the Model:

- Sends the generated text to the next stage in the system.

ASR to Sign:


Input text
Preprocess text
Syntax Analysis
Convert to ISL grammer
For each word, is it found is ISL dictionary?
No
Skip Word
Yes
Get ISL sign for current word
Generate ISL animation
Display

Description of the mentioned components in brief:

Start:

- This is the entry point of the system.
- Initializes the process of conversion.

Input text:

- Input is given in the form of plain text.
- This raw data will be converted to ISL.

Preprocess text:

- Cleans and prepares text for preprocessing.
- Tasks may include:
    1) Removing punctuation
    2) Converting to lowercase
    3) Tokenizing words

Syntax Analysis:

- It analyzes the structure of the input text.
- Identifies different parts of the text like noun, verb, adjective etc.
- Helps rearrange words according to ISL grammar.

Convert to ISL Grammer:

- Transforms English to ISL grammar.
- This step ensures that the sentence is in correct Is structure.

Check if word exists in ISL Dictionary:

- It checks if the word mapping exists.
- Decision Point:
    1) Yes → Word is acceptable; get ISL sign.
    2) No → Word is skipped

Skip Word:

- If a word is not found, it is ignored.
- Prevents errors.

Get ISL sign for current word:

- Retrieve the ISL sign for the word from the ISL dictionary.

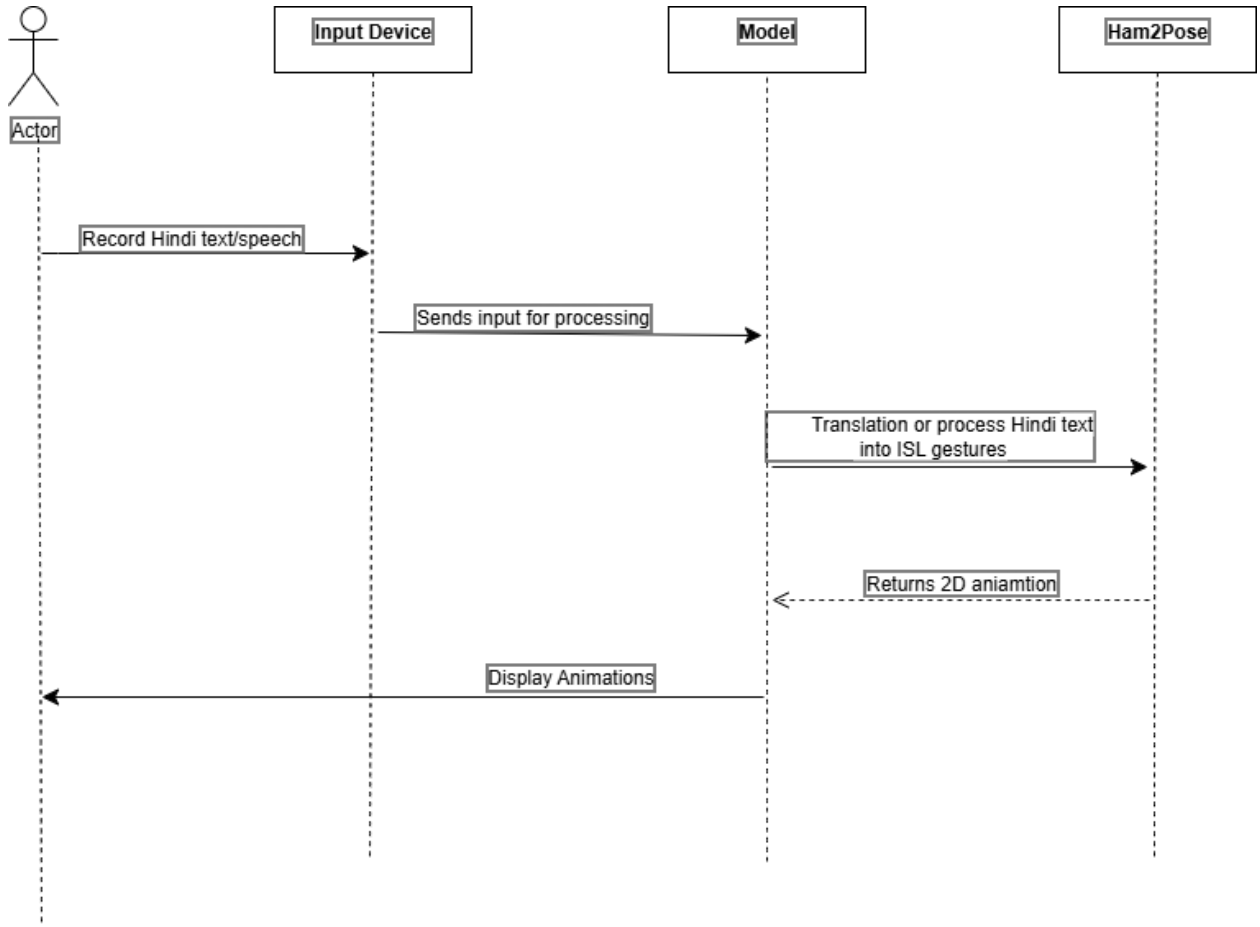
Generate ISL animation:

- Combines ISL signs into a continuous 2D animation.

Display:

- The final output is shown to the user.
- Allows the user to view the translated signs.

3.5. Sequence Diagram



Description of the mentioned components in brief:

A. **Actor:**

   Represents the end-user who speaks Hindi text/speech.

B. **Input Device:**

   Captures the user's speech or text.

C. **Model:**

   Handles speech-to-text conversion and manages translation logic.

Sequence Flow:

Step 1- Record Hindi text/speech

The user speaks in Hindi. The input device records the voice.

Step 2- Sends input for processing

The recorded Hindi speech (raw input) is sent to the Model for processing.

Step 3- Preprocessed Text

Inside the Model, the speech is converted into text (Speech-to-Text), cleaned (preprocessing), and formatted.

Step 4- Model Call

Then the m2m100 model is used to translate or process this Hindi text into ISL representation.

Step 5: Returns 2D Animation

The system returns a 2D animated video/sign language avatar that shows the ISL signs for the given text.

Step 6: Display Animations

The final animated video is displayed back to the user, showing the translation in sign language.

4. Coding(Modules)

Modules used are:

1. **Speech Recognition:**

   Algorithm:

   STEP 1: Initialize speech recognizer

   STEP 2: Activate microphone and capture audio

   STEP 3: Convert audio signal into digital format.

   STEP 4: Send audio to speech recognition engine.

   STEP 5: Recognize spoken Hindi and convert into text.

   STEP 6: If recognition is successful, then return text.

   STEP 7: Else, return error message

   This function captures Hindi speech using a microphone and converts it into text. It uses a speech recognition engine. In the implementation, the function voice_to_text() listens to the user and processes the audio. If the speech is unclear, or if there is a network issue, appropriate error messages are returned.

2. Text Preprocessing

   Algorithm:

   STEP 1: Accept Hindi text input from user

   STEP 2: Check if input is empty

   STEP 3: Normalize text

   STEP 4: Split text into sentences using sentence tokenizer

   STEP 5: Convert sentences into tokens/words

   STEP 6: Store processed tokens in a list

   STEP 7: Pass processed text to translation module.

   This module prepares raw Hindi input for further processing. It ensures that the input is clean, structured and ready for further translation. The system uses a function to divide the given input into meaningful and sensible sentences.

   This module improves the translation accuracy by processing the input text at the sentence level.

   The output of this module is a structured set of tokens that can be easily mapped to its corresponding ISL representations.

3. Hindi to English translation

   Algorithm:

   STEP 1: Accept processed Hindi sentences

   STEP 2: Load Translation Model- MTrans

   STEP 3: for each sentence:

   Convert Hindi sentence to english

   STEP 4: Store translated sentence in a list

   STEP 5: return english output

   This module converts Hindi text to English using a translation model. This step is particularly important because the ISL gloss generation module works with English based NLP. The output is a list of English sentences which serve as input for the next stage.

4. Gloss Generation (English to ISL)

   Algorithm:

   Step 1: Prepare the NLP Tool

   Start by using a language processing model, such as spaCy, to break the English sentence into individual words. The model also identifies the grammatical role of each word, whether it is a noun, verb, adjective, or adverb.

   Step 2: Classify Words

   Go through each word in the sentence and determine its type. Some words indicate time (e.g., “today,” “morning”), some indicate location (e.g., “school,” “home”), and the rest are main content words that depict the meaning of sentence, such as nouns, verbs, adjectives, adverbs, and numbers.

   Step 3: Remove Unimportant Words

   Remove words that do not add meaning, such as punctuation or common function words like “is,” “the,” and “and.” Only keep words that contribute to the meaning of the sentence.

   Step 4: Convert Words to Gloss Form

   For the remaining words, take their base form (lemma) and write them in uppercase. Optionally, verbs, adjectives, or adverbs can be marked with suffixes like -V or -ADJ if needed for clarity.

   Step 5: Reorder Words

   Arrange the words in a logical order used in ISL glosses: time → location → main

content. This ensures the resulting gloss is easy to read and makes sense in ISL structure.

Step 6: Combine into a Gloss Sentence

Finally, join all the words together with spaces to form a single gloss sentence.

Example:

Input : “I am going to school”

Output: “I GO SCHOOL”

5. Video Mapping and Animation Generation

Algorithm:

STEP 1: Accept ISL gloss sentence as input

STEP 2: Split gloss into individual words

STEP 3: for each word:

A. Convert word to lowercase

B. Search corresponding video file in the dataset

C. If found, add video to the clip list

STEP 4: If no video found,

Return error message

STEP 5: Merge all video clips together

STEP 6: Generate the final animation video

STEP 7: Display the final output video.

This module converts ISL gloss to visual 2D animation. The system retrieves these videos and merges them into a single continuous video using video processing libraries. It ensures that the output is visually understandable and follows a correct sequence of gestures.

6. **User Interfaces module**

Algorithm:

STEP 1: Display user interface

STEP 2: Allow user to choose input method(Text/Speech)

STEP 3: if speech is selected,

Call speech recognition module.

STEP 4: if text is selected,

Accept Hindi input

STEP 5: Send input to preprocessing module

STEP 6: Perform translation and gloss generation

STEP 7: Display gloss output

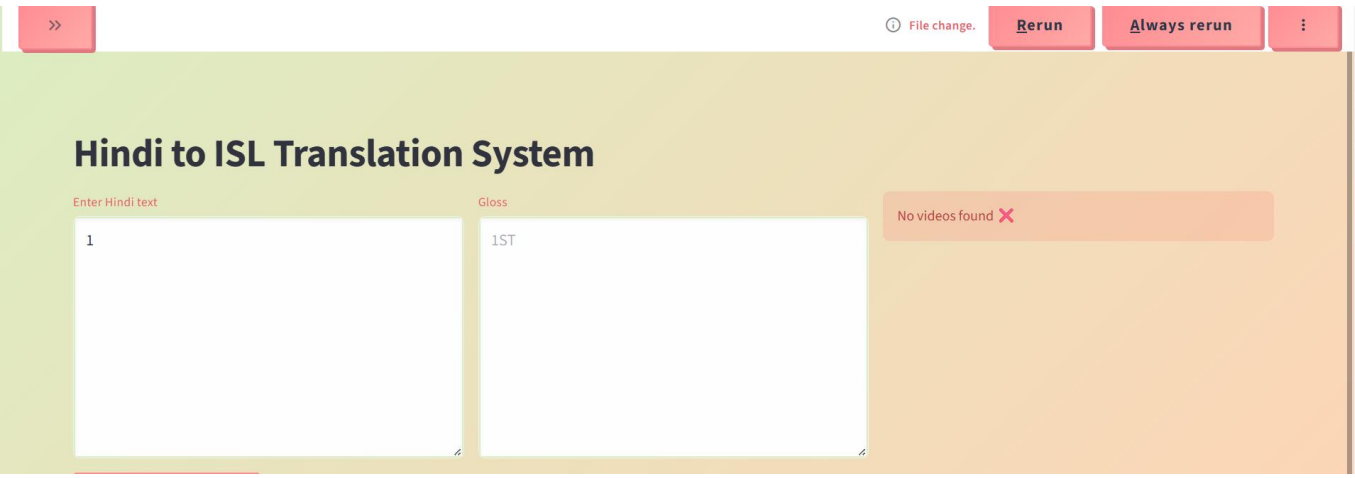
STEP 8: Generate and display animation

STEP 9: Handle errors if any step fails

This module manages the interaction between the end user and the system. It is implemented using Streamlit and provides an interactive interface. Users can either type Hindi text or speak through the microphone. The system processes the input step by step and displaces both the gloss and the 2D animated output. The module also provides feedback, such as warning and error messages, to improve user experience.

5. Testing(Test cases)

A. Numeric Values:



One recurring issue identified during the testing phase is the system's struggle to accurately detect and process numeric values within the Hindi input.

While the NLP toolkit handles standard words well, it often fails to recognize digits and causing them to be skipped or mislabeled during the transition to gloss. This is a significant hurdle because, in Indian Sign Language (ISL), numbers are vital for establishing context, especially when following the TIME → LOCATION format.

To resolve this issue, we need to refine the system's training to ensure it prioritizes numbers and doesn't strip them away and it know the correct translation both in gloss format as well as Animation format. Improving this detection is essential for making the final output both accurate and useful for the end user.

B. Hindi Numeric text

D. Changed form or certain words

Another issue found is that the system sometimes changes the specific word forms in Hindi.

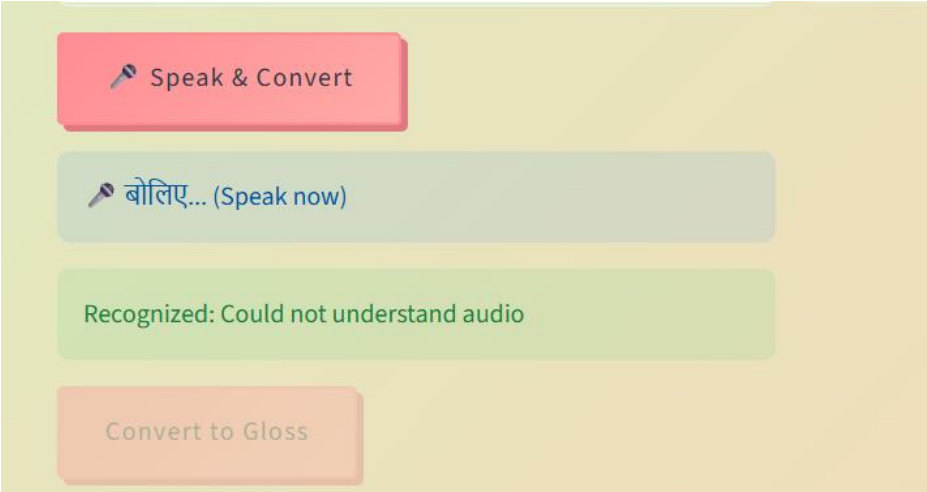
For example, when "नही" is entered, the glossing process might label it as "NOT" instead of "NO," which changes the nuance of the sign.

This is especially problematic with family terms like "brother" or "husband," where the system picks the wrong root word based on how the sentence is structured.

This creates more problem in limited database when a certain word form is not found like as no is more usual, the database stores “no” rather than “Not”. Since ISL relies on very specific glosses, these small

errors can completely flip the meaning of a conversation. this need to tighten up the mapping logic to make sure the system recognizes the correct base form every time.

D.Incorrect Audio recognition-



Here is a breakdown of the speech recognition issues:

- **Audio Sensitivity**: The system is currently very sensitive to how someone speaks; if there's even a little background noise or a slight accent, the recognition starts to fall apart.

- The Domino Effect: When the speech-to-text engine mishears a Hindi word, the entire pipeline is disrupted. The system ends up glossing a word that was never actually said.

- Lack of Natural Flow: Right now, the tool doesn't handle casual, everyday conversation well. It only seems to work if the user speaks unnaturally slow and hits every syllable perfectly.

- Critical Translation Errors: Because Indian Sign Language relies on precise keywords, a single misheard audio cue can completely change the meaning of the final output.

- Need for Robustness: We need to improve the system's ability to filter out ambient noise and recognize a wider variety of Hindi dialects.

- User Frustration: If a user has to repeat themselves five times just to get one sentence translated, the system loses its practical value in real-time settings.

6 . User Interfaces

This section provides a description of the software's user interface (UI) design, focusing on simplicity, accessibility, and ease of use.

Description of the user interface

The user interface will be a clean, intuitive web-based application. The design will be minimalistic to ensure that users can perform translations with the fewest steps possible. The layout will be responsive, adapting to different screen sizes for use on desktops, tablets, and mobile devices.

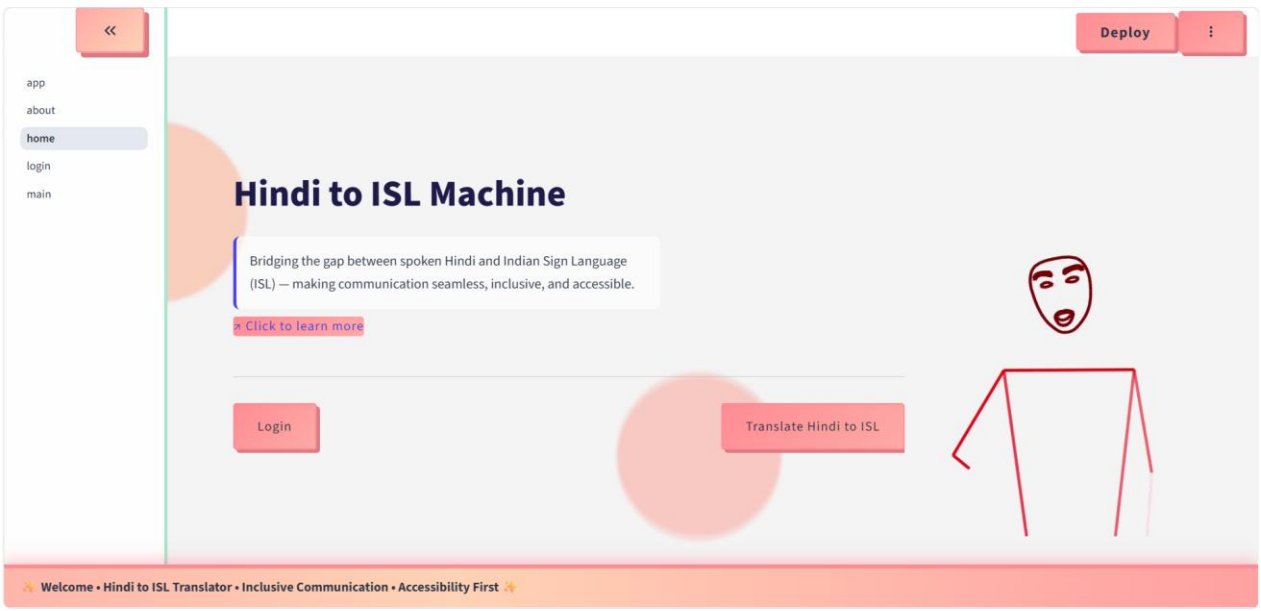
The main screen will be divided into three logical sections:

1. **Input Section:** A prominent area where users can either type Hindi text or activate their microphone for speech input.
2. **Control Section:** Contains the primary action button, "Translate," to initiate the process.
3. **Output Section:** A large, clear area dedicated to displaying the resulting 2D animation of the ISL gesture.

Accessibility will be a key consideration, with high-contrast colors, clear fonts, and support for screen readers.

Screen image:

1) home



Contains a small description of the website at the top right, with welcome character at right corner

The page also contains a bottom bar for more information.

It has a “click to know more button” right below text that points to about page of

ISL translation system.

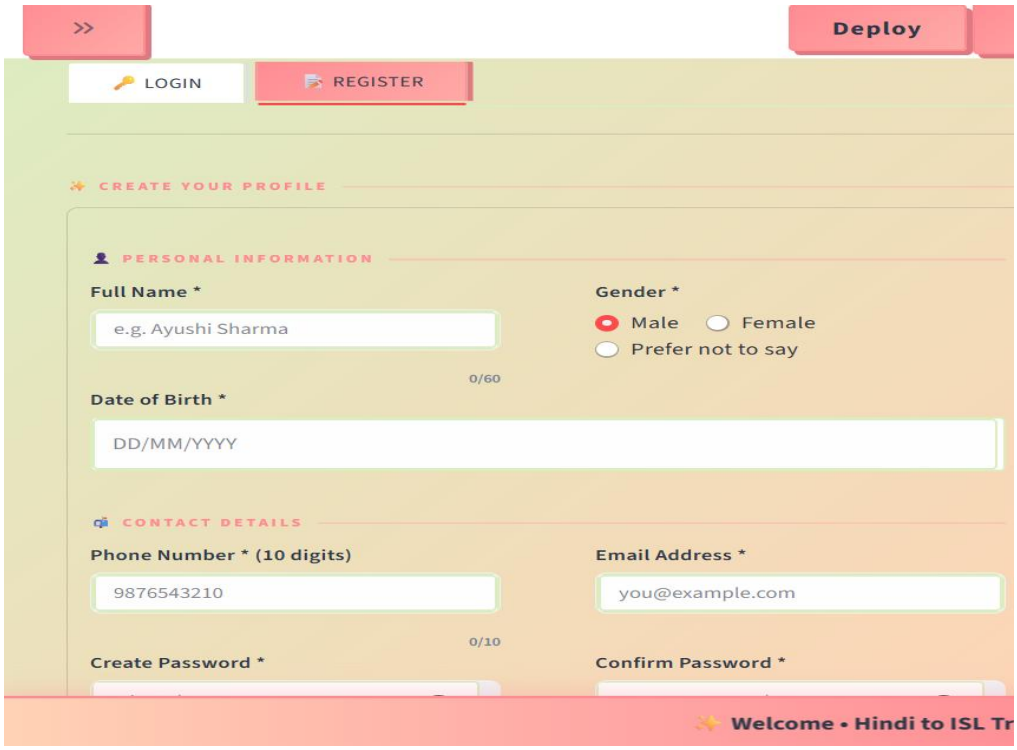
The page contains login button on the left bottom and translate button on right side.

Objects and Actions:

- **Object: about Button**
  - **Purpose:** To switch to about.py page.
- **Object: Translate Button**
  - **Purpose:** to switch to main.py page,

    For hindi to ISL Translation.
  - **Properties:** A clearly labeled button. It is valid for logged in or without a logged in account.

2) **login/Registration**



Registration page is to create an account for the website , by filling some required and some not mandatory fields

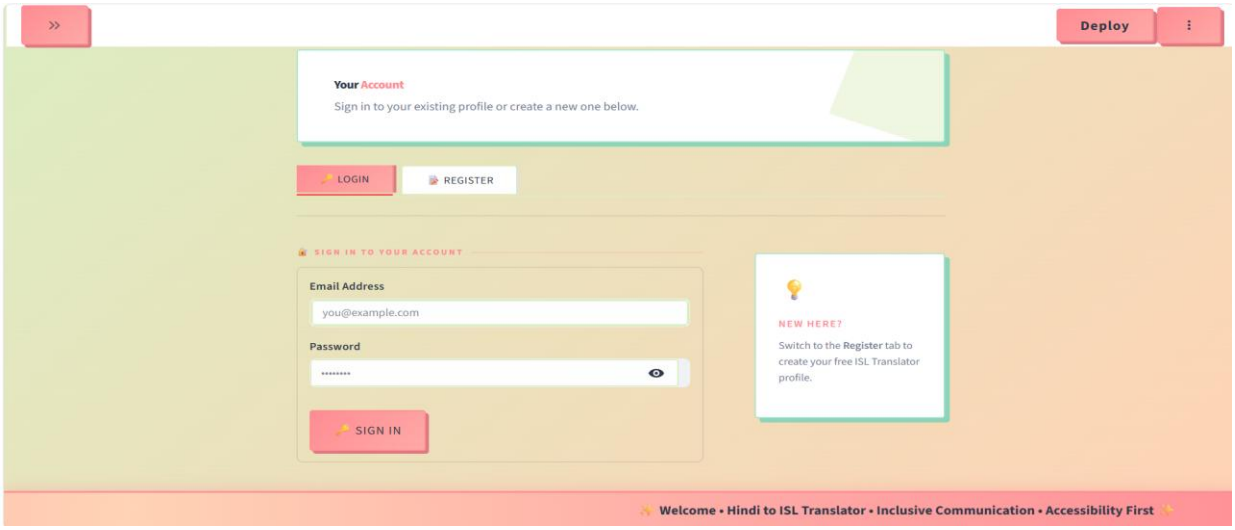
That includes -

Personal details- Full name, gender, DOB

Contact details- Phone number, valid email address, password to login later from that account

ISL & preference- Preferred language(english/hindi), Notification preference, ISL experience etc.

Optional short bio.

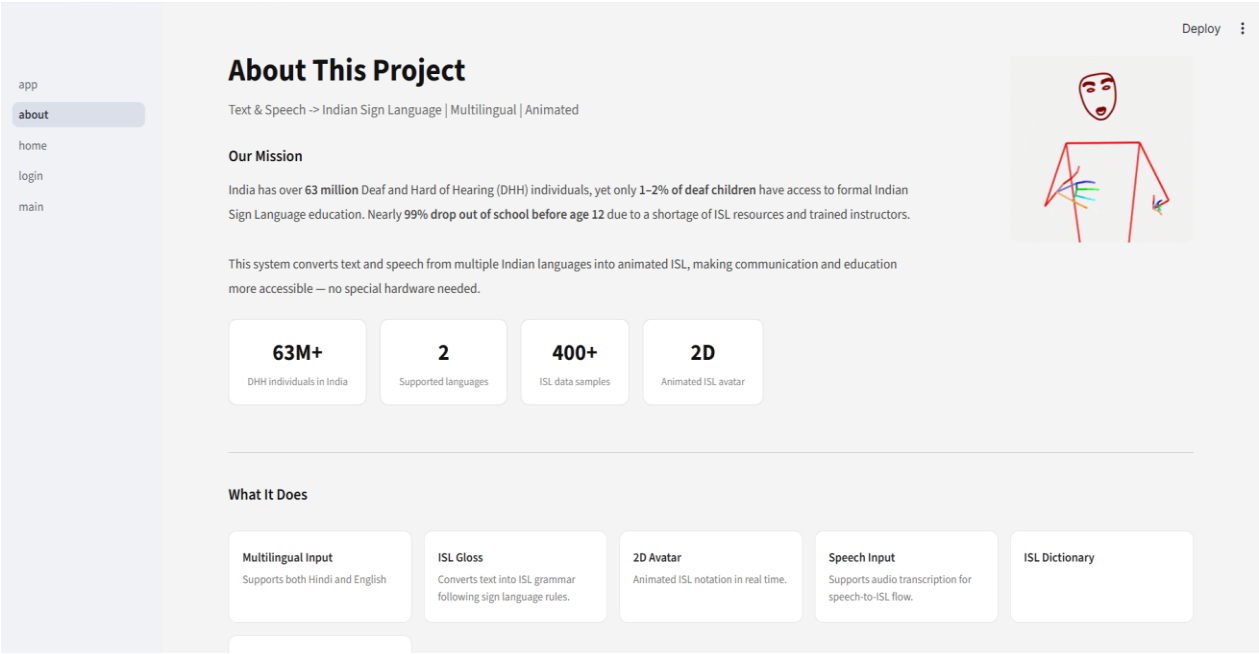


Login page: contains the fields of email and password to log in to pre occurring account

Both the log in and registration pages are connected to the database.

Once the login/registration is complete, the screen shows up the button to translate page

3) About page:



About page is a static page with mentions the objective of this project and the community it is built for

It discusses the importance of communication and the barrier in between for people of DHH community (Deaf and Hard of Hearing community)

It discusses the statistics and data collected.

This page further mentions about the working and mechanism of the website,

It discusses about-

What It Does

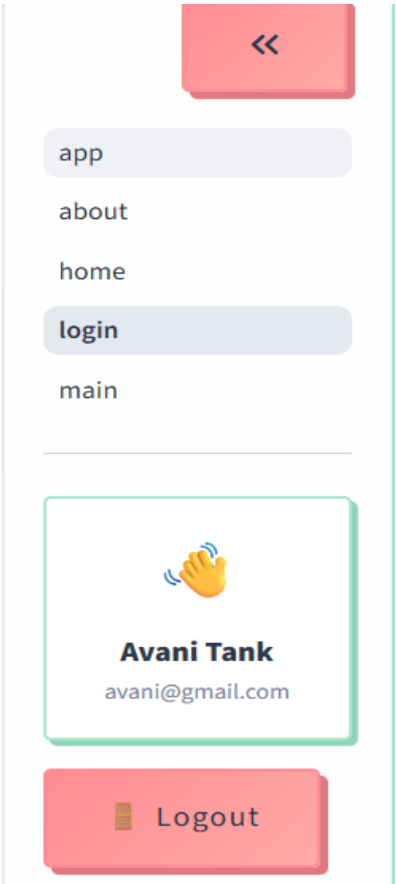
- Multilingual Input-
  Supports both Hindi and English
- ISL Gloss
  Converts text into ISL grammar following sign language rules.
- 2D Avatar
  Animated ISL notation in real time.
- Speech Input
  Supports audio transcription for speech-to-ISL flow.
- ISL Dictionary
- Education-Ready
  Built for schools, hospitals & government portals.

The about page further talks about-

How it works-

Text / Audio -> NLP Cleaning -> Multilingual Translation -> ISL Gloss -> Animated Dataset -> 2D Avatar

4) **Side Bar:**



Purpose- To help users quickly navigate between different pages of the website

<u>Objects and Actions</u>

Object: Sidebar panel

- Purpose: Contains all navigation options to different pages
- Properties: can expand or collapse
- Events/Responses:
    - On Toggle: Opens or closes the sidebar

Object: Navigation

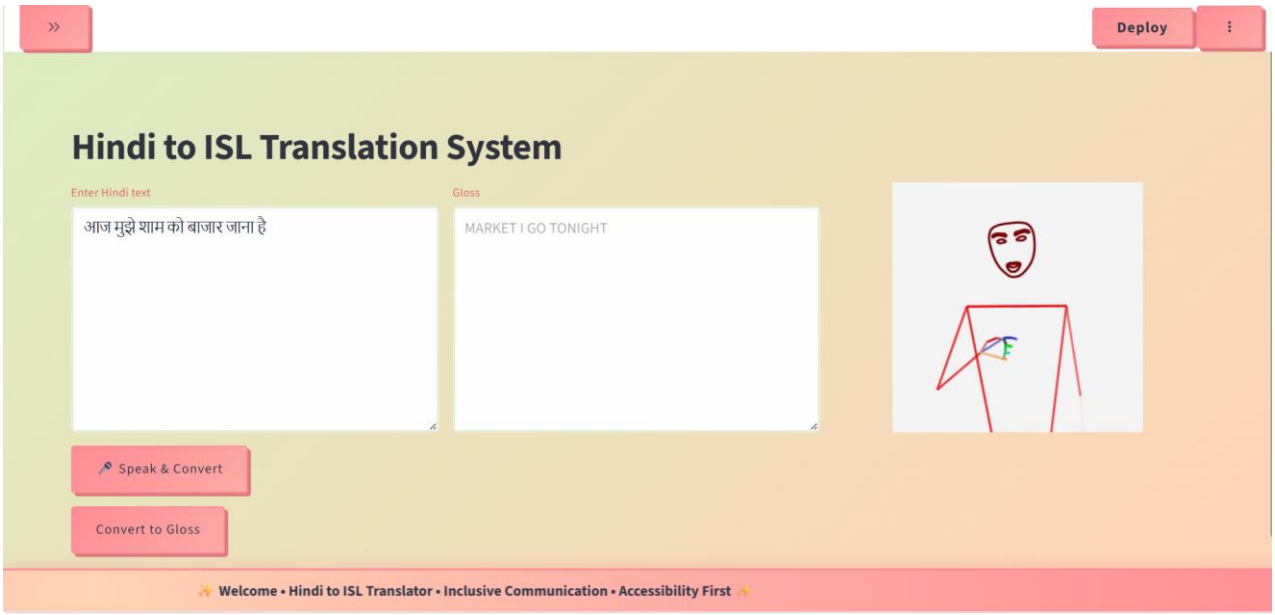
Includes items like:

- Home
- About
- Translate Page
- **Events/Responses:**
    - On Click, Redirects to the selected page

Object: Logout Button

- **Purpose:** Log the user out

5)translation page:

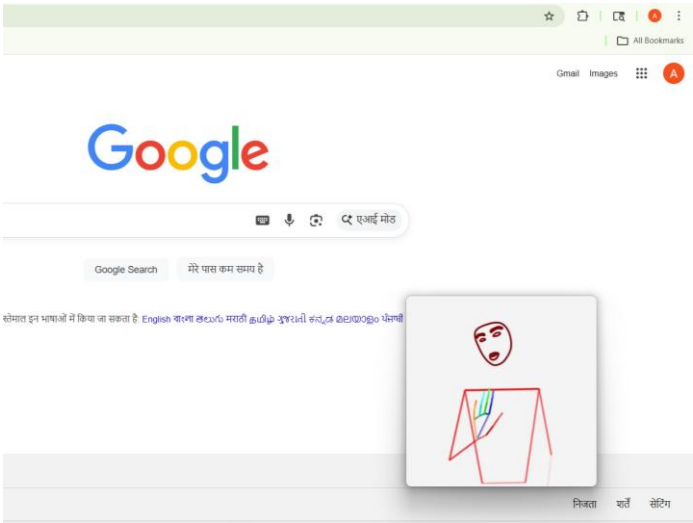


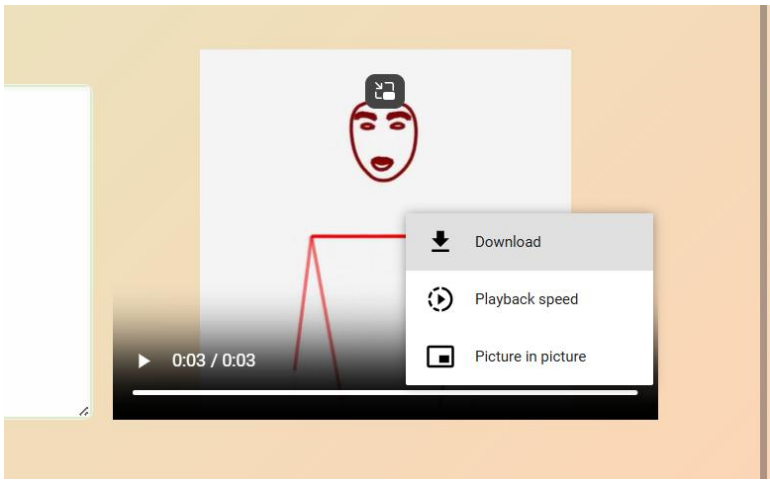
<u>Objects and Actions</u>

This subsection identifies all screen objects and the actions associated with them.

- **Object: Input Text Area**
    - **Purpose:** To allow users to type or paste Hindi text.
    - **Data Type:** String.
    - **Properties:** Multi-line text field with a placeholder prompt.
    - **Events/Responses:**
        - **On Key Press:** Accepts Hindi characters.

- **Object: Microphone Button**
  - **Purpose:** To enable speech-to-text input.
  - **Properties:** Icon button, toggles between 'active' and 'inactive' states.
  - **Events/Responses:**
    - **On Click (Inactive State):** Changes to 'active' state, begins recording audio from the user's microphone. The icon may be animated to show its listening.
    - **On Click (Active State):** Stops recording and sends the audio data for processing.
- **Object: Translate Button**
  - **Purpose:** To submit the user's input (either text or recorded speech) for translation.
  - **Properties:** A clearly labeled button. It may be disabled until valid input is provided.
  - **Events/Responses:**
    - **On Click:** Sends the content of the Input Text Area or the recorded audio to the backend. A loading indicator may appear while waiting for the response.





- **Object: Output Display Area**
  - **Purpose:** To display the final ISL animation.
  - **Properties:** A video player component embedded in the page.
  - **Events/Responses:**
    - **On Response Success:** The area becomes visible and automatically plays the received animation video.
    - **On Animation End:** The video may loop or show replay controls.
    - **On Response Error:** Displays an error message (e.g., "Translation not found").

The system utilises the .mp4 format, this ensure compatibility all across the platform and all devices, The website support **Picture-in-Picture (PiP)** feature, it can be minimized and move out from tab Hence the window can be minimized and the audio functionality remains active.that acts as a real time speech recognition and translation provider.

Hence can be functioning in background with audio feature, or speech recognition in real time for various platform like Youtube, or any other media

To enhance user experience and give them full control over their viewing experience, the interface includes several essential tools:

- **Playback Speed Controls:** Users can speed up or slow down the.mp4 translation animation to suit their needs.
- **Offline Accessibility :** An in built **download** option that user allows for local storage and offline viewing
- **replay** functionality for review of key segments.

7. Appendices

Abbreviations and Acronyms:

- **SDS (Software Design Specification):**
  A detailed document describing the design, structure, and functionality of the software system.
- **SRS (Software Requirements Specification):**
- A detailed document describing the functional and non-functional requirements of a software system. It defines what the system should do, its features, constraints, and expected behavior, serving as a foundation for system design and development.
- **ISL (Indian Sign Language):**
  A visual language used by the deaf and hard-of-hearing community in India.
- **API (Application Programming Interface):**
  A set of protocols and tools that allow communication between different software components.
- **ASR (Automatic Speech Recognition):**
  Technology used to convert spoken language into text.
- **DHH (Deaf and Hard of Hearing):**
  Refers to individuals with hearing impairments.

Supporting Information:

- The system uses the **iSign dataset** as the primary source for mapping Hindi text to ISL gestures.
- The **Ham2Pose model** is used for generating 2D animated ISL gestures.
- The system follows a **modular architecture**, consisting of input processing, preprocessing, translation, and animation modules.
- Temporary data structures are used instead of a traditional database, ensuring faster processing and reduced storage dependency.
- The system is designed to be scalable and can support future enhancements such as additional languages and improved grammar handling.

8. References

- R. Shalev, A. Shafran, O. Barzilay, and A. A. Gutfreund, “Ham2Pose: Animating Sign Language Notation into Pose Sequences,” in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2023.
- P. Jalote, An Integrated Approach to Software Engineering, 3rd ed. New York, NY, USA: Springer, 2005.
- R. S. Pressman, Software Engineering: A Practitioner's Approach, 7th ed. New York, NY, USA: McGraw-Hill, 2010.